

[Wiki »](#)

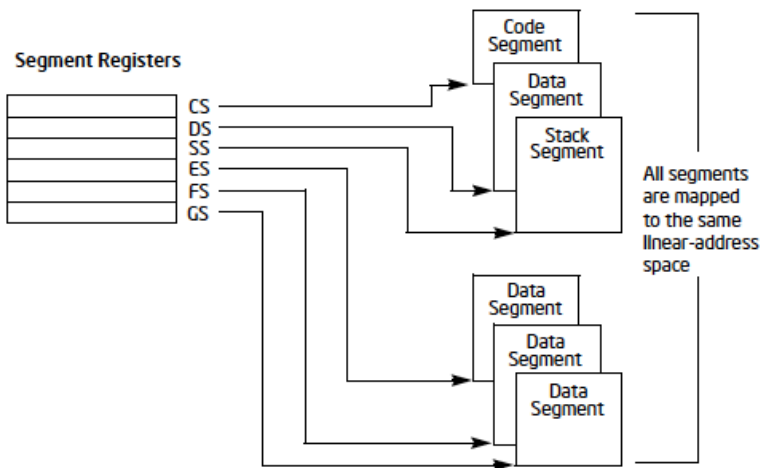
Лабораторная работа №2 ¶

Внимание! Данная лабораторная работа содержит множество отсылок к документации Intel Software Developer Manual (SDM). Актуальную версию сборного документа из 4 томов можно скачать по [ссылке](#).

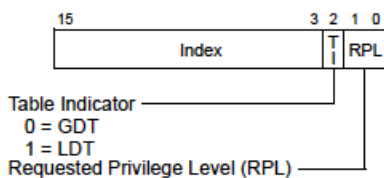
Работа в различных режимах процессора

Современные реализации архитектуры x86 позволяют процессору работать в разных режимах работы. В зависимости от текущего активного режима меняется набор команд и их поведение, а также доступные для использования ресурсы. Переключение между режимами выполняется программно, и как на разных стадиях инициализации платформы, так и в разное время работы операционной системы, процессор может работать в разных режимах.

Перед тем как перейти к рассмотрению режимов работы процессора, необходимо понять, как устроена сегментная адресация в архитектуре x86. Подробно о сегментной адресации можно прочесть в Intel SDM в разделе 3.4.2 Segment Registers подраздела 3.4.1 Basic Program Execution Registers первого тома и разделах 3.2 Using Segments третьего тома. Любой используемый машинным кодом адрес принадлежит некоторому сегменту, который описывает где в оперативной памяти этот адрес находится и какими свойствами он обладает. Для выбора сегмента в инструкциях явно или неявно используют сегментные регистры процессора. Адрес текущей исполняемой инструкции принадлежит кодовому сегменту (описывается сегментным регистром `cs`), адрес на стеке принадлежит сегменту стека (`ss`), данные в памяти — сегменту данных (`ds`), если иной сегмент не указан явно (например, `gs`, `fs`, `es`, см. 3.7.4 Specifying a Segment Selector).

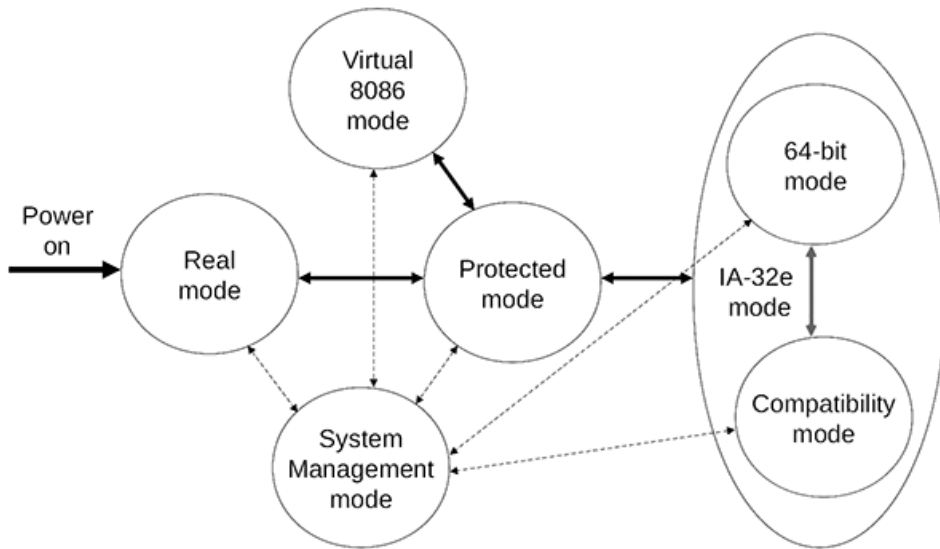


Сегментный регистр внутри процессора состоит из двух частей: видимой и скрытой. Значение сегментного регистра является видимой частью, биты 3:15 которой содержат индекс элемента в таблице в оперативной памяти, который описывает данный сегмент. В момент обновления значения сегментного регистра процессор загружает из памяти данный элемент и сохраняет его в скрытой части сегментного регистра. Данная таблица называется таблицей дескрипторов и может быть либо глобальной (`gdt`), либо локальной (`ldt`), в зависимости от бита 2 в видимой части сегментного регистра. Значения, загруженные в сегментный регистр из данной таблицы, в том числе определяют текущий режим работы процессора.



Хотя некоторые режимы работы процессора требуют использования виртуальной памяти, которая в архитектуре x86 реализована посредством страничной адресации, для данной лабораторной работы объяснения принципов её работы не требуются, за исключением факта её наличия и эквивалентности физического и виртуального адресов в нашей конфигурации. Работа с виртуальной памятью будет позднее рассмотрена в следующих лабораторных работах.

Доступные на текущий момент режимы процессора и возможности прямого переключения между ними изображены на рисунке ниже:



- **Реальный режим** (Real mode) процессора является старейшим режимом с 16-битным набором команд и 20-битной адресацией. Первые процессоры Intel, начиная с Intel 8086 и вплоть до 80286, могли использовать исключительно реальный режим работы. Находясь в реальном режиме работы, есть возможность перейти либо в защищённый режим, либо в режим системного управления.
- **Защищённый режим** (Protected mode) процессора является основным режимом для 32-битных процессоров Intel, появившийся в Intel 80286. Данный режим использует 32-битный набор команд, позволяет адресовать больше памяти (32-битная адресация без PAE) и поддержку виртуальной памяти, позволяющую реализовывать изоляцию задач в операционной системе.
- **Режим системного управления** (System Management mode) является прозрачным механизмом изоляции платформы от операционной системы, который позволяет разработчику платформы реализовывать слой управления питанием, организовывать безопасный доступ к критичным для платформы устройствам, таким как SPI FLASH, или производить эмуляцию устаревших интерфейсов, таких как PS/2 или VGA (UEFI Compatibility Support Module).
- **Длинный режим** (Long mode, IA-32e mode) является основным режимом работы для 64-битных процессоров Intel. Данный режим использует 64-битный набор команд и использует 64-битную (или 48-битную в ранних версиях) адресацию памяти. В числе ключевых отличий от защищённого режима является обязательное использование виртуальной памяти, отказ от сегментной адресации и упрощённая относительная адресация для поддержки PIC/PIE кода.

Кроме основных режимов работы процессора также выделяются вспомогательные режимы или подрежимы работы:

- **Нереальный режим** (Unreal mode) работы процессора — возможность процессоров Intel 80286 и новее использовать 32-битные сегменты, обеспечивая адресацию памяти за пределами 1 мегабайта.
- **Режим виртуального 8086** (Virtual 8086 mode) — аналог реального режима, позволяющий использовать 16-битный набор команд после перехода в защищённый режим.
- **Режим совместимости** (Compatibility mode) — аналог защищённого режима, позволяющий использовать 32-битный набор команд после перехода в длинный режим.

Переход в длинный режим

Хотя основные реализации UEFI прошивок x86 используют 64-битную DXE фазу, для встраиваемых систем на базе x86 и виртуальных машин можно встретить 32-битную реализацию. Выбор архитектуры прошивки для запуска JOS в виртуальной машине QEMU выполняется посредством переменной окружения ARCHS. Значение X64 (по умолчанию) означает 64-битный запуск, значение IA32 производит запуск в 32-битном режиме.

Запустите JOS с 32-битной прошивкой OVMF с помощью команды `ARCHS=IA32 make qemu`. Если вы всё сделали правильно, то запуск операционной системы остановится на строчке `JOS: KernelCallGate: 0x618C10`. Это связано с тем, что ядро JOS ожидает передачи управления в 64-битном режиме, однако код для перехода из 32-битного режима в 64-битный в загрузчике ещё не был реализован.

Отредактируйте файл `LoaderPkg/Loader/IA32/Transition.nasm` загрузчика JOS, добавив в него код для перехода из 32-битного защищённого режима в 64-битный длинный режим. Подробно переход в 64-битный режим описан в разделе 9.8.5 Initializing IA-32e Mode Intel SDM. Краткий пересказ данного процесса также приведён ниже:

1. Отключите использование виртуальной памяти (Paging) в регистре `CR0`. Формат таблицы страниц виртуальной памяти для длинного и защищённого режима отличается, поэтому переход напрямую в длинный режим без отключения виртуальной памяти невозможен. Подробнее см. в 4.1.2 Paging-Mode Enabling Intel SDM.
2. Загрузите новую глобальную таблицу дескрипторов, в которой будут содержаться дескрипторы с поддержкой длинного режима и обновите сегментные регистры в значения, соответствующие сегментам кода и данных для защищённого режима. Обратите внимание, что обновление регистра `cs` невозможно обычной командой `mov`, и для этого используются команды `far jmp` или `far ret`.
3. Активируйте поддержку расширения физического адреса (PAE) и глобальных страниц (PGE) в регистре `CR4`, необходимые для возможности использования нового формата таблицы страниц виртуальной памяти.
4. Обновите регистр адреса таблицы страниц виртуальной памяти (`CR3`) на новую таблицу страниц, которая была передана в функцию. Обратите внимание, что виртуальная память не будет использоваться до момента её повторной активации в регистре `CR0`.
5. Активируйте поддержку длинного режима (LME) и бита защиты от исполнения памяти (NXE) в MSR регистре `EFER`.
6. Активируйте использование виртуальной памяти (Paging) в регистре `CR0`. При необходимости выставите биты мониторинга сопроцессора (MON), защиты памяти (WP) и выравнивания (ALIGN).
7. Обновите сегментные регистры для использования длинного режима.

Если вы всё сделали правильно, то JOS будет запускаться с помощью команды `ARCHS=IA32 make qemu` аналогично запуску без `ARCHS=IA32`.

Добавление команды монитора

Взаимодействие ядра с пользователем в данный момент осуществляется с помощью монитора — интерактивной программы управления. Для тестирования ядро JOS настроено таким образом, что оно выводит данные не только на экран, но и в последовательный порт, данные из которого QEMU использует в качестве стандартного вывода; ввод данных также осуществляется как с клавиатуры, так и из последовательного порта. Таким образом, команды монитора можно вводить как в окне QEMU, так и в окне терминала. В данный момент заданы только две команды монитора: `help` и `kerninfo`.

Просмотрите файл `kern/monitor.c`. Как задаются команды монитора и как происходит их вызов? Каким образом происходит вывод текста в консоль? Добавьте свою команду, которая выводит в консоль произвольный текст.

Стек

Для работы с локальными переменными и для вызова функций используется область памяти, называемая стеком. Указатель стека (регистр `RSP`) указывает на вершину стека — начало области стека, которая используется в настоящее время. Вся память ниже этого указателя является свободной. При добавлении значения в стек указатель стека уменьшается, а затем нужное значение записывается в новую вершину стека. При извлечении значения происходит чтение вершины стека, а затем его указатель увеличивается. В 64-битном режиме стек может содержать только 64-битные значения, поэтому `RSP` всегда кратно 8. Регистр `RSP` используется различными инструкциями `x86`, такими как `call`.

Регистр `RBP` (указатель базы), в отличие от `RSP`, связан со стеком программным образом. При входе в `C`-функцию код пролога функции обычно сохраняет указатель базы предыдущей функции, записывая его в стек, а затем копирует текущее значение `RSP` в `RBP` на время выполнения функции. Если все функции в программе следуют этому правилу, то в любой момент выполнения программы можно проследить порядок вызова функций через стек, следуя цепочке сохраненных указателей `RBP`, и определить, какая последовательность вызовов функций привела к достижению данного места выполнения программы. Эта возможность может быть особенно полезна, например, когда та или иная функция вызывает `assert` или `panic` из-за неправильных аргументов, но вы не уверены, какая именно функция передала эти аргументы. Трассировка стека позволяет найти эту функцию.

```

+-----+
| ret %rip | выше черты стек вызывающей функции
+-----+
| saved %rbp | ниже черты стек вызываемой функции
%rbp -> +-----+
| local      |
| vars       |
%rsp -> +-----+
```

Определите, где происходит инициализация стека, и где стек расположен в памяти. Как ядро резервирует область памяти для стека? Какое место в этой области памяти является началом стека?

Чтобы ознакомиться с [соглашением о вызовах C в System V на x86-64](#), найдите адрес функции `test_backtrace` в файле `obj/kern/kernel.asm`, установите там точку останова и посмотрите, что происходит при каждом вызове функции после загрузки ядра. Сколько 64-разрядных слов каждый уровень `test_backtrace` добавляет в стек, что это за слова?

Приведенное выше упражнение должно дать вам информацию, необходимую для реализации функции трассировки стека `mon_backtrace()`. Прототип для этой функции уже находится в `kern/monitor.c`. Вы можете написать функцию на `C` без использования ассемблера — для этого может оказаться полезной функция `read_rbp()` в `inc/x86.h`. Вы также должны добавить новую функцию в список команд монитора, чтобы она могла быть вызвана пользователем.

Функция трассировки должна отображать данные в следующем формате:

```

Stack backtrace:
rbp 0000008041616f00 rip 00000080416041ef
rbp 0000008041616fd0 rip 0000008041600294
rbp 0000008041616ff0 rip 0000008041600015
...
```

Первая строка соответствует выполняемой в данный момент функции (`mon_backtrace`), вторая — функции, которая вызвала `mon_backtrace` и так далее. Изучив файл `kern/entry.S`, вы найдете простой способ определить момент, когда нужно остановиться.

В каждой строке значение `RBP` соответствует базовому указателю на область стека, используемую данной функцией, то есть положению указателя стека сразу после того, как функция начала работать и код пролога функции установил базовый указатель. Значение `RIP` является указателем на адрес возврата: адресом, по которому будет передано управление после выхода из функции. Указатель на адрес возврата обычно указывает на следующую инструкцию после `call` (почему?).

Почему код трассировки не может обнаружить, сколько аргументов было на самом деле? Как это можно исправить?

Реализуйте функцию трассировки, как описано выше. Используйте тот же формат, что и в примере, иначе скрипт проверки не сработает. Если вы считаете, что функция работает правильно, запустите `JOSLLVM=0` (или `JOSLLVM=1`) `make grade`, чтобы увидеть, соответствует ли вывод функции тому, что ожидает скрипт проверки, и исправьте его, если не соответствует.

Отладочная информация

Начиная с данной лабораторной работы для облегчения отладки написанного кода имеется доступ к `UndefinedBehaviorSanitizer` на уровне ядра, с инструкцией по использованию которого можно ознакомиться по [ссылке](#). Все решения лабораторных работ должны быть проверены с помощью `UndefinedBehaviorSanitizer`.

В данный момент функция трассировки выводит адреса функций в стеке, которые привели к выполнению `mon_backtrace()`. Однако на практике удобно знать имена функций, соответствующих этим адресам — например, чтобы узнать, в каких функциях может находиться ошибка, приводящая к краху ядра. Для этого используется отладочная информация, которая создается компилятором и компоновщиком при сборке двоичного файла (в данном случае — образа ядра). Отладочная информация может храниться в самом исполняемом файле или отдельно от него. Существуют различные форматы отладочных данных, такие как `STABS`, `COFF`, `DWARF`. В `JOS` используется формат `DWARF`, а отладочные данные хранятся в самом образе ядра.

Для работы с отладочными данными можно использовать функцию `debuginfo_rip()`, которая ищет значение `RIP` в таблице символов и возвращает отладочную информацию для этого адреса. Эта функция определена в `kern/kdebug.c`.

Измените функцию трассировки стека для отображения для каждого значения `RIP` соответствующих имени функции, имени исходного файла и номера строки.

Откуда берутся значения `Debug*` в `debuginfo_rip`? Чтобы найти ответ, попробуйте сделать следующие вещи:

1. Выполните `objdump -h obj/kern/kernel`.
2. Узнайте зачем в `GNUmakefile` используются `gcc` флаги `-g` и `-gpubnames`.

Дополните реализацию функции `debuginfo_rip` вызовом `line_for_address` для нахождения номера строки в файле.

Дополните реализацию функции `dwarf_read_abbrev_entry` в файле `kern/dwarf.c` разбором случая формы записи `DW_FORM_block2` в таблице аббревиатур по аналогии со случаем `DW_FORM_block4`.

Добавьте команду монитора `backtrace`, вызывающую функцию `mon_backtrace`, и расширьте реализацию `mon_backtrace` вызовом `debuginfo_rip` для вывода номера строки для каждого кадра стека в следующем виде:

```
K> backtrace
Stack backtrace:
 rbp 0000008041616f00 rip 00000080416041ef
   kern/monitor.c:124: monitor+429
 rbp 0000008041616fd0 rip 0000008041600294
   kern/init.c:123: i386_init+155
 rbp 0000008041616ff0 rip 0000008041600015
   kern/entry.S:21: <unknown>+0
K>
```

Для каждого значения `RIP` выводится имя файла и номер строки, за ними следуют имя функции и смещение значения `RIP` от начала этой функции (например, `monitor+432` означает, что `RIP` указывает на 432-й байт от начала функции `monitor`).

Обратите внимание: чтобы скрипт проверки работал правильно, имя функции и файла должны выводиться на отдельной строке.

Вы можете обнаружить, что некоторые вызовы функций (например, `glibc()`) отсутствуют в трассировке. Это происходит из-за встраивания их вызовов компилятором. Кроме того, из-за других оптимизаций, вносимых компилятором, могут выводиться неправильные номера строк. Если удалить опцию компилятора `-O1` в `GNUmakefile`, трассировка станет более правильной (но ядро будет работать несколько медленнее).

По окончании выполнения работы следует создать в репозитории новую ветку `working-lab2` и сохранить в ней внесенные изменения. После этого нужно добавить удаленный репозиторий для сохранения результатов по предоставленному преподавателем адресу и отправить в него эту ветку, например:

```
git checkout -b working-lab2
git add <...>
git commit -m "Lab 2 done."
git remote add myrepo http://sed.ispras.ru/git/<name>-osprac # здесь <name> - ваш логин на sed.ispras.ru
git push myrepo working-lab2
```

Updated by [Alexey Khoroshilov](#) 5 months ago · 1 revisions